claude-windows / 50e27c80-8f59-4d8f-8a14-fdbddb9fbb3a

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\50e27c80-8f59-4d8f-8a14-fdbddb9fbb3a\subagents\workflows\wf_ddb0b3fd-b40\agent-a3038e99338c3bd6e.jsonl`
- SHA-256: `e0584e460dc15d4cdf27a78db50a4d3491d6c3738402c21a016a50229413cddf`
- Source modified: `2026-07-07T04:16:38+00:00`
- Imported at: `2026-07-08T16:00:26+00:00`
- project: `wf_ddb0b3fd-b40`
- session_id: `50e27c80-8f59-4d8f-8a14-fdbddb9fbb3a`

Transcript

1. user (2026-07-07T04:14:53.572Z)

You are reviewing an UNCOMMITTED working diff implementing issue #506 (GPU-resident WASB inference) in the repo at C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer. The full diff is at C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/50e27c80-8f59-4d8f-8a14-fdbddb9fbb3a/scratchpad/506_diff.patch (read it). The new test file tests/test_wasb_nvdec_resident.py is untracked (not in the diff) ? read it directly from the repo.

Ground truth you must compare against:
- The VALIDATED reference implementation: C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/scratch/gpu_resident/exp_t2_full.py (the GpuFrontend class + its grid build ARE what was validated on a real L4: F1 0.574 ? baseline 0.582 on the hard clip). The production port must be semantically faithful to it.
- WASB-SBDT interfaces (not in this repo; trust these facts): TracknetV2Detector.run_tensor(imgs, affine_mats) moves imgs .to('cuda') itself and calls postprocessor.run(preds, affine_mats) which does affine_mats[scale].cpu().numpy() then indexes [i] per batch item; ImageDataset (eval) returns (imgs_t, hms_t, trans_outputs_inv, xys, visis, img_paths) where trans_outputs_inv is {scale: np.float64 [2,3]}; per-frame preprocessing is PIL RGB -> cv2.warpAffine(trans_input, (512,288), INTER_LINEAR) -> ToTensor -> Normalize(ImageNet); get_transform uses c=(w/2,h/2), s=max(h,w); OnlineTracker uses np.Inf; WASB detector.py calls torch.load(model_path) with no weights_only arg.
- The CPU decode path (run(), run_video_streaming) must be BEHAVIOR-UNCHANGED on the WSL torch-1.11 + numpy<1.24 env (byte-parity guarantee), except that torch>=2.6 envs get the weights_only + np.Inf shims.

Report ONLY defects you can ground in the actual code you read ? cite file:line. No style nits, no speculative maybes without a concrete failure scenario. Return findings as structured output.

A reviewer claims this defect in the #506 diff:
- file: deploy/digitalocean/gpu_setup.sh
- line: 75
- summary: The trailing `|| true` on `python -m pip install -q hydra-core omegaconf pandas opencv-python-headless pillow scipy tqdm pyyaml matplotlib numpy` silently swallows failure of what the adjacent comment now declares to be 'the (L4-validated) dependency closure' ? the SOLE source of WASB's mandatory runtime deps (the requirements.txt fallback was removed in this diff). The [6/6] sanity check only validates torch + PyNvVideoCodec, not hydra/pandas/cv2, and the parallel Dockerfile line correctly has no `|| true` (build fails loud).
- failure scenario: A transient PyPI/resolver failure during setup makes the pip install fail; `|| true` masks it under `set -euo pipefail`, the script prints 'native WASB env ready', and the first paid GPU inference run dies mid-job with ModuleNotFoundError: hydra (or pandas/cv2) inside wasb_infer.py ? exactly the fail-late mode the repo's fail-loud convention forbids.
- severity: minor

Adversarially VERIFY it: read the actual code (and the validated reference / WASB interface facts above) and try to REFUTE the claim. It is only real if the concrete failure scenario

actually happens in the code as written. Default to isReal=false if the scenario cannot occur,
is already guarded, or rests on a misreading.

2. user (2026-07-07T04:15:00.602Z)

```
1       #!/usr/bin/env bash
2       #
3       # Native (no-WSL) WASB shuttle-detector setup for a Linux GPU box ? the M3.5 enabler for
4       # real trajectory extraction. Replicates docs/TRACKNET_WSL_SETUP.md's WSL conda env
5       # *natively* on Linux. Run AFTER ./bootstrap.sh --gpu (which provides CUDA torch for the
6       # main venv); this builds a SEPARATE conda env for WASB ? never co-installed with the
7       # indexer's torch (framework conflict).
8       #
9       # #506 modernized stack (validated end-to-end on a real L4, 2026-07-06): py3.10 +
10      # torch 2.6.0+cu124 (WASB HRNet ports clean; cpu F1 ? the torch-1.11 baseline) +
11      # PyNvVideoCodec for the GPU-resident decode backend (indexing.wasb.decode_backend=
12      # nvdec_resident, ~1.7×). Step [2/6] stages the NVDEC/NVENC driver userspace libs
13      # PyNvVideoCodec needs (compute-only driver images ship neither).
14      #
15      # Requires an NVIDIA GPU + driver (the DO `gpu-*-base` image ships it).
16      #
17      # ? WASB-C3: the pretrained badminton weights' license / source-dataset provenance is
NOT
18      # verified here. Fine for internal extraction + Gen-0 bootstrapping; clear it before
sharing
19      # any derived trajectories or model. See docs/DECOUPLED_COMPUTE/06.
20      set -euo pipefail
21
22      MODELS_DIR="${RALLY_MODELS_DIR:-$HOME/models}"
23      CONDA_DIR="${CONDA_DIR:-$HOME/miniconda3}"
24      WASB_REPO="$MODELS_DIR/WASB-SBDT"
25      mkdir -p "$MODELS_DIR"
26
27      echo "[gpu] [1/6] miniconda (native Linux)"
28      if [ ! -x "$CONDA_DIR/bin/conda" ]; then
29        curl -sSL https://repo.anaconda.com/miniconda/Miniconda3-latest-Linux-x86_64.sh -o
/tmp/mc.sh
30        bash /tmp/mc.sh -b -p "$CONDA_DIR" && rm -f /tmp/mc.sh
31      fi
32      # shellcheck disable=SC1091
33      source "$CONDA_DIR/etc/profile.d/conda.sh"
34
35      echo "[gpu] [2/6] NVDEC/NVENC driver userspace libs (#506: PyNvVideoCodec needs BOTH)"
36      # Compute-only driver images (GCE DLVM, DO gpu-*-base) ship neither libnvcuvid.so.1 nor
37      # libnvidia-encode.so.1. Extract them from the .run matching the RUNNING driver so the
38      # userspace/kernel versions can never skew (see the #506 / gpu-vm-setup-gotchas notes).
39      if [ ! -f /usr/lib/x86_64-linux-gnu/libnvcuvid.so.1 ] || [ ! -f /usr/lib/x86_64-linux-
gnu/libnvidia-encode.so.1 ]; then
40        DRV="$(nvidia-smi --query-gpu=driver_version --format=csv,noheader | head -1)"
41        echo "[gpu]       extracting libnvcuvid + libnvidia-encode from driver $DRV ..."
42        curl -fsSL -o /tmp/drv.run "https://us.download.nvidia.com/tesla/$DRV/NVIDIA-
Linux-x86_64-$DRV.run" \
43          || curl -fsSL -o /tmp/drv.run
"https://us.download.nvidia.com/XFree86/Linux-x86_64/$DRV/NVIDIA-Linux-x86_64-$DRV.run"
44        sh /tmp/drv.run --extract-only --target /tmp/drv-extract >/dev/null
45        sudo cp -a /tmp/drv-extract/libnvcuvid.so.* /tmp/drv-extract/libnvidia-encode.so.*
/usr/lib/x86_64-linux-gnu/
46        ( cd /usr/lib/x86_64-linux-gnu \
47          && sudo ln -sf "libnvcuvid.so.$DRV" libnvcuvid.so.1 \
48          && sudo ln -sf "libnvidia-encode.so.$DRV" libnvidia-encode.so.1 \
49          && sudo ldconfig )
50        rm -rf /tmp/drv-extract /tmp/drv.run
51      else
52        echo "[gpu]       already present"
53      fi
```

```
54
55      # Modern conda BLOCKS non-interactive `conda create` until the defaults-channel Terms of
Service
56      # are accepted (CondaToSNonInteractiveError). Accept them so the env build doesn't fail
on a fresh
57      # box. (Observed on a GCP Deep-Learning-VM image, 2026-06-20.)
58      conda tos accept --override-channels --channel https://repo.anaconda.com/pkgs/main
>/dev/null 2>&1 || true
59      conda tos accept --override-channels --channel https://repo.anaconda.com/pkgs/r
>/dev/null 2>&1 || true
60
61      echo "[gpu] [3/6] WASB-SBDT repo"
62      if [ ! -d "$WASB_REPO/.git" ]; then
63        git clone --depth 1 https://github.com/nttcom/WASB-SBDT.git "$WASB_REPO"
64      fi
65
66      echo "[gpu] [4/6] conda env 'wasb' (py3.10 + torch 2.6.0+cu124 + PyNvVideoCodec ? #506
stack)"
67      conda env list | grep -q '/envs/wasb$' || conda create -y -q -n wasb python=3.10
68      conda activate wasb
69      python -m pip install -q torch==2.6.0 torchvision==0.21.0 \
70        --index-url https://download.pytorch.org/whl/cu124
71      python -m pip install -q PyNvVideoCodec
72      # WASB-SBDT ships NO requirements.txt: src/ imports hydra, pandas, etc. ? this explicit
list is
73      # the L4-validated dependency closure (#506). numpy is UNPINNED (the old "numpy<1.24"
was the
74      # torch-1.11 ABI pin); wasb_infer.py shims the NumPy-2.0 np.Inf removal for WASB's
tracker.
75      python -m pip install -q hydra-core omegaconf pandas opencv-python-headless pillow scipy
tqdm pyyaml matplotlib numpy || true
76
77      echo "[gpu] [5/6] pretrained badminton weights"
78      WEIGHTS="$WASB_REPO/pretrained_weights/wasb_badminton_best.pth.tar"
79      if [ -f "$WEIGHTS" ]; then
80        echo "[gpu]       found $WEIGHTS"
81      else
82        echo "[gpu]       !! MISSING $WEIGHTS ? fetch wasb_badminton_best.pth.tar
(Drive/bucket) before extraction."
83      fi
84
85      echo "[gpu] [6/6] GPU sanity check (in the wasb env)"
86      python - <<'PY'
87      import torch
88      ok = torch.cuda.is_available()
89      print("[gpu]       torch", torch.__version__, "| cuda_available", ok,
90            "| device", (torch.cuda.get_device_name(0) if ok else "CPU"))
91      assert ok, "CUDA not available in the wasb env ? check the GPU driver / base image"
92      try:
93          import PyNvVideoCodec as pnvc
94          print("[gpu]       PyNvVideoCodec", getattr(pnvc, "__version__", "?"), "OK")
95      except Exception as e:  # nvdec_resident unavailable; the cpu decode path still works
96          print("[gpu]       !! PyNvVideoCodec import failed:", repr(e)[:150])
97      PY
98
99      echo "[gpu] native WASB env ready.  Activate with:  source
$CONDA_DIR/etc/profile.d/conda.sh && conda activate wasb"
100     echo "[gpu] (the LinuxNativeRunner ? backend/pipeline/detectors ? drives extraction once
it lands; M3.5.)"
101
```

3. user (2026-07-07T04:15:02.824Z)

```
403:+               f"/ gpu_setup.sh stage them). Fall back to decode_backend=cpu."
687: # native GPU box, deploy/digitalocean/gpu_setup.sh):
```

```
765:diff --git a/deploy/digitalocean/gpu_setup.sh b/deploy/digitalocean/gpu_setup.sh
767:--- a/deploy/digitalocean/gpu_setup.sh
768:+++ b/deploy/digitalocean/gpu_setup.sh
```

4. user (2026-07-07T04:15:21.151Z)

```
-        hydra-core omegaconf pandas opencv-python-headless pillow scipy tqdm pyyaml matplotlib
"numpy<1.24"
+        hydra-core omegaconf pandas opencv-python-headless pillow scipy tqdm pyyaml matplotlib
numpy

 # --- Main application env ---
 WORKDIR /app
diff --git a/deploy/digitalocean/gpu_setup.sh b/deploy/digitalocean/gpu_setup.sh
index 8d47e49..a89cc1b 100755
--- a/deploy/digitalocean/gpu_setup.sh
+++ b/deploy/digitalocean/gpu_setup.sh
@@ -3,8 +3,14 @@
 # Native (no-WSL) WASB shuttle-detector setup for a Linux GPU box ? the M3.5 enabler for
 # real trajectory extraction. Replicates docs/TRACKNET_WSL_SETUP.md's WSL conda env
 # *natively* on Linux. Run AFTER ./bootstrap.sh --gpu (which provides CUDA torch for the
-# main venv); this builds a SEPARATE conda env for WASB (torch 1.11) ? never co-installed
-# with the indexer's torch (framework conflict).
+# main venv); this builds a SEPARATE conda env for WASB ? never co-installed with the
+# indexer's torch (framework conflict).
+#
+# #506 modernized stack (validated end-to-end on a real L4, 2026-07-06): py3.10 +
+# torch 2.6.0+cu124 (WASB HRNet ports clean; cpu F1 ? the torch-1.11 baseline) +
+# PyNvVideoCodec for the GPU-resident decode backend (indexing.wasb.decode_backend=
+# nvdec_resident, ~1.7×). Step [2/6] stages the NVDEC/NVENC driver userspace libs
+# PyNvVideoCodec needs (compute-only driver images ship neither).
 #
 # Requires an NVIDIA GPU + driver (the DO `gpu-*-base` image ships it).
 #
@@ -18,7 +24,7 @@ CONDA_DIR="${CONDA_DIR:-$HOME/miniconda3}"
 WASB_REPO="$MODELS_DIR/WASB-SBDT"
 mkdir -p "$MODELS_DIR"

-echo "[gpu] [1/5] miniconda (native Linux)"
+echo "[gpu] [1/6] miniconda (native Linux)"
 if [ ! -x "$CONDA_DIR/bin/conda" ]; then
   curl -sSL https://repo.anaconda.com/miniconda/Miniconda3-latest-Linux-x86_64.sh -o /tmp/mc.sh
   bash /tmp/mc.sh -b -p "$CONDA_DIR" && rm -f /tmp/mc.sh
@@ -26,29 +32,49 @@ fi
 # shellcheck disable=SC1091
 source "$CONDA_DIR/etc/profile.d/conda.sh"

+echo "[gpu] [2/6] NVDEC/NVENC driver userspace libs (#506: PyNvVideoCodec needs BOTH)"
+# Compute-only driver images (GCE DLVM, DO gpu-*-base) ship neither libnvcuvid.so.1 nor
+# libnvidia-encode.so.1. Extract them from the .run matching the RUNNING driver so the
+# userspace/kernel versions can never skew (see the #506 / gpu-vm-setup-gotchas notes).
+if [ ! -f /usr/lib/x86_64-linux-gnu/libnvcuvid.so.1 ] || [ ! -f /usr/lib/x86_64-linux-
gnu/libnvidia-encode.so.1 ]; then
+  DRV="$(nvidia-smi --query-gpu=driver_version --format=csv,noheader | head -1)"
+  echo "[gpu]       extracting libnvcuvid + libnvidia-encode from driver $DRV ..."
+  curl -fsSL -o /tmp/drv.run "https://us.download.nvidia.com/tesla/$DRV/NVIDIA-
Linux-x86_64-$DRV.run" \
+    || curl -fsSL -o /tmp/drv.run
"https://us.download.nvidia.com/XFree86/Linux-x86_64/$DRV/NVIDIA-Linux-x86_64-$DRV.run"
+  sh /tmp/drv.run --extract-only --target /tmp/drv-extract >/dev/null
+  sudo cp -a /tmp/drv-extract/libnvcuvid.so.* /tmp/drv-extract/libnvidia-encode.so.*
/usr/lib/x86_64-linux-gnu/
+  ( cd /usr/lib/x86_64-linux-gnu \
+    && sudo ln -sf "libnvcuvid.so.$DRV" libnvcuvid.so.1 \
+    && sudo ln -sf "libnvidia-encode.so.$DRV" libnvidia-encode.so.1 \
```

```
+    && sudo ldconfig )
+  rm -rf /tmp/drv-extract /tmp/drv.run
+else
+  echo "[gpu]       already present"
+fi
+
 # Modern conda BLOCKS non-interactive `conda create` until the defaults-channel Terms of
Service
 # are accepted (CondaToSNonInteractiveError). Accept them so the env build doesn't fail on a
fresh
 # box. (Observed on a GCP Deep-Learning-VM image, 2026-06-20.)
 conda tos accept --override-channels --channel https://repo.anaconda.com/pkgs/main >/dev/null
2>&1 || true
 conda tos accept --override-channels --channel https://repo.anaconda.com/pkgs/r    >/dev/null
2>&1 || true

-echo "[gpu] [2/5] WASB-SBDT repo"
+echo "[gpu] [3/6] WASB-SBDT repo"
 if [ ! -d "$WASB_REPO/.git" ]; then
   git clone --depth 1 https://github.com/nttcom/WASB-SBDT.git "$WASB_REPO"
 fi

-echo "[gpu] [3/5] conda env 'wasb' (py3.8 + torch 1.11.0+cu113 ? the verified WASB stack)"
-conda env list | grep -q '/envs/wasb$' || conda create -y -q -n wasb python=3.8
+echo "[gpu] [4/6] conda env 'wasb' (py3.10 + torch 2.6.0+cu124 + PyNvVideoCodec ? #506 stack)"
+conda env list | grep -q '/envs/wasb$' || conda create -y -q -n wasb python=3.10
 conda activate wasb
-python -m pip install -q torch==1.11.0+cu113 torchvision==0.12.0+cu113 \
-  --extra-index-url https://download.pytorch.org/whl/cu113
-[ -f "$WASB_REPO/requirements.txt" ] && python -m pip install -q -r
"$WASB_REPO/requirements.txt" || true
-# WASB-SBDT's requirements.txt is INCOMPLETE: src/ imports hydra, pandas, etc. that it omits,
so
-# `import wasb_infer` deps fail without these. (Found on the GCP L4 port, 2026-06-20.) numpy is
-# pinned <1.24 for torch-1.11 ABI compatibility.
-python -m pip install -q hydra-core omegaconf pandas opencv-python-headless pillow scipy tqdm
pyyaml matplotlib "numpy<1.24" || true
+python -m pip install -q torch==2.6.0 torchvision==0.21.0 \
+  --index-url https://download.pytorch.org/whl/cu124
+python -m pip install -q PyNvVideoCodec
+# WASB-SBDT ships NO requirements.txt: src/ imports hydra, pandas, etc. ? this explicit list is
+# the L4-validated dependency closure (#506). numpy is UNPINNED (the old "numpy<1.24" was the
+# torch-1.11 ABI pin); wasb_infer.py shims the NumPy-2.0 np.Inf removal for WASB's tracker.
+python -m pip install -q hydra-core omegaconf pandas opencv-python-headless pillow scipy tqdm
pyyaml matplotlib numpy || true

-echo "[gpu] [4/5] pretrained badminton weights"
+echo "[gpu] [5/6] pretrained badminton weights"
 WEIGHTS="$WASB_REPO/pretrained_weights/wasb_badminton_best.pth.tar"
 if [ -f "$WEIGHTS" ]; then
   echo "[gpu]       found $WEIGHTS"
@@ -56,13 +82,18 @@ else
   echo "[gpu]       !! MISSING $WEIGHTS ? fetch wasb_badminton_best.pth.tar (Drive/bucket)
before extraction."
 fi

-echo "[gpu] [5/5] GPU sanity check (in the wasb env)"
+echo "[gpu] [6/6] GPU sanity check (in the wasb env)"
 python - <<'PY'
 import torch
 ok = torch.cuda.is_available()
 print("[gpu]       torch", torch.__version__, "| cuda_available", ok,
       "| device", (torch.cuda.get_device_name(0) if ok else "CPU"))
 assert ok, "CUDA not available in the wasb env ? check the GPU driver / base image"
+try:
```

```
+    import PyNvVideoCodec as pnvc
+    print("[gpu]       PyNvVideoCodec", getattr(pnvc, "__version__", "?"), "OK")
+except Exception as e:  # nvdec_resident unavailable; the cpu decode path still works
+    print("[gpu]       !! PyNvVideoCodec import failed:", repr(e)[:150])
 PY

 echo "[gpu] native WASB env ready.  Activate with:  source $CONDA_DIR/etc/profile.d/conda.sh &&
conda activate wasb"
diff --git a/docs/CLOUD_GPU_DRYRUN_GUIDE.md b/docs/CLOUD_GPU_DRYRUN_GUIDE.md
index 8e263b4..6021064 100644
--- a/docs/CLOUD_GPU_DRYRUN_GUIDE.md
+++ b/docs/CLOUD_GPU_DRYRUN_GUIDE.md
@@ -93,8 +93,9 @@ Watch it finish (look for `SETUP_DONE` and `torch ... cuda_available True`):
 gcloud compute ssh rally-gpu --zone="$ZONE" --command='tail -n 20 ~/setup.log'
 ```

-> ? You're ready when the log shows the WASB env built with `torch 1.11.0+cu113 |
cuda_available True
-> | device NVIDIA L4` and `SETUP_DONE`. (The 5.8 MB detector weights are now staged.)
+> ? You're ready when the log shows the WASB env built with `torch 2.6.0+cu124 | cuda_available
True
+> | device NVIDIA L4` (plus a `PyNvVideoCodec ... OK` line ? the #506 GPU-resident decode dep)
and
+> `SETUP_DONE`. (The 5.8 MB detector weights are now staged.)

 ---

diff --git a/docs/PACKAGING_PLAN.md b/docs/PACKAGING_PLAN.md
index 9822606..5a2873a 100644
--- a/docs/PACKAGING_PLAN.md
+++ b/docs/PACKAGING_PLAN.md
@@ -20,7 +20,7 @@

 ## 0. TL;DR
-Ship the engine as **ONE pip-installable artifact whose behaviour is selected entirely by the
already-merged decoupled-compute seam** (`deployment.profile` / `compute_target` /
Storage+Compute backends), fronted by a friendly launcher + a Windows installer ? **not** a
frozen monolith, **not** an Electron/Tauri shell. The hard constraint that forces scope: torch
is deliberately out of `pyproject` (needs `--index-url`) **and** native WASB needs a *second,
incompatible* torch env (conda py3.8 + torch 1.11+cu113) shelled out via `$WASB_PYTHON`
`[verified native_wasb_runner.py:89]` ? so no single venv/freeze can deliver the GPU-quality
path. **The honest v1 is a LIGHT tier** (Gemini + motion + cpu-mock, *zero* torch/GPU) that
preserves CUJ-1..5/8; the native-WASB GPU path (CUJ-6/7) ships **owner-only** until two blockers
clear (WASB-C3 weight provenance + a reproducible truly-local config). **LIGHT is the FIRST
shippable increment, not the terminal scope ? the sole completion goal (north star, L10) is the
FULL quality path: native WASB + BYO training + in-app annotation.** Delivery is **cross-
platform (Windows/macOS/Linux): the same local webserver powers the UI on all three, with NO
native component** (L9). **Five small engine prerequisites must land before any packaging push**
(§7 P0). The single most actionable finding: **all mutable state is CWD-relative today and
`--setup`/server disagree on `config.json`** ? a silent data-loss footgun that P0 must fix
first.
```

**5. user (2026-07-07T04:15:21.506Z)**

```
15:+   # runner only ? needs the torch-2.x + PyNvVideoCodec WASB env
(deploy/cloudrun/Dockerfile)
402:+   f"and libnvcuvid/libnvidia-encode on the box (deploy/cloudrun/Dockerfile "
681:diff --git a/deploy/cloudrun/Dockerfile b/deploy/cloudrun/Dockerfile
683:--- a/deploy/cloudrun/Dockerfile
684:+++ b/deploy/cloudrun/Dockerfile
911: | **CLOUD / server** | GCP deploy + BYO-GPU-VM | the `deploy/cloudrun/Dockerfile` image
(micromamba-ified); cloud-serving + gcs; behind Cloudflare Access | `gcloud run jobs create` per
`deploy/cloudrun/README.md`, or `deploy/gcp` VM scripts. Bind?`0.0.0.0` (config-gated) + edge-
auth mandatory. |
```
```

6. user (2026-07-07T04:16:01.484Z)

```
diff --git a/deploy/cloudrun/Dockerfile b/deploy/cloudrun/Dockerfile
index 836ffb7..aee76a0 100644
--- a/deploy/cloudrun/Dockerfile
+++ b/deploy/cloudrun/Dockerfile
@@ -8,13 +8,33 @@
 # Two python stacks, deliberately separate (a framework conflict otherwise ? same split as the
 # native GPU box, deploy/digitalocean/gpu_setup.sh):
 #   * the MAIN app env (this repo + ffmpeg) drives the pipeline + stitching;
-#   * a conda `wasb` env (py3.8 + torch 1.11.0+cu113) runs the WASB shuttle detector as a
subprocess
-#     (the native runner shells out to $WASB_PYTHON), so the app's torch never co-installs with
it.
+#   * a conda `wasb` env (py3.10 + torch 2.6.0+cu124 + PyNvVideoCodec ? the #506 modernized
stack,
+#     validated on a real L4: cpu-decode F1 0.576 ? the torch-1.11 baseline 0.582) runs the
WASB
+#     shuttle detector as a subprocess (the native runner shells out to $WASB_PYTHON), so the
app's
+#     torch never co-installs with it.
 #
 # ? WASB-C3: the pretrained badminton weights are academic-data-trained ? provenance NOT
cleared for
 #   commercial sharing. They are NOT baked into this image; stage them at runtime (a bucket
fetch or a
 #   mount) and point $WASB_WEIGHTS at them. Resolve WASB-C3 before any public, non-owner
serving.

+# Driver userspace libs for NVDEC/NVENC (#506): GPU hosts inject the COMPUTE driver stack but
not
+# the video-codec libs ? no libnvcuvid.so.1, no libnvidia-encode.so.1 ? and PyNvVideoCodec
needs
+# BOTH (it initializes NVENC even for decode-only). Extract the pair from the matching
datacenter
+# driver .run in a throwaway stage. Keep NVIDIA_DRIVER_VERSION aligned with the GPU fleet's
driver
+# (580.159.03 = the validated L4 combo); these userspace libs talk to the host-injected
libcuda,
+# so a large version skew can break decode ? the golden gate on the real image is the check.
+ARG NVIDIA_DRIVER_VERSION=580.159.03
+FROM nvidia/cuda:12.4.1-runtime-ubuntu22.04 AS nvlibs
+ARG NVIDIA_DRIVER_VERSION
+RUN apt-get update && apt-get install -y --no-install-recommends curl ca-certificates \
+    && rm -rf /var/lib/apt/lists/* \
+    && curl -fsSL -o /tmp/drv.run \
+        "https://us.download.nvidia.com/tesla/${NVIDIA_DRIVER_VERSION}/NVIDIA-
Linux-x86_64-${NVIDIA_DRIVER_VERSION}.run" \
+    && sh /tmp/drv.run --extract-only --target /tmp/drv \
+    && mkdir -p /nvlibs \
+    && cp -a /tmp/drv/libnvcuvid.so.* /tmp/drv/libnvidia-encode.so.* /nvlibs/ \
+    && rm -rf /tmp/drv /tmp/drv.run
+
 FROM nvidia/cuda:12.4.1-runtime-ubuntu22.04

 ENV DEBIAN_FRONTEND=noninteractive \
@@ -29,22 +49,38 @@ RUN apt-get update && apt-get install -y --no-install-recommends \
         python3 python3-pip python3-venv \
     && rm -rf /var/lib/apt/lists/*

+# NVDEC/NVENC userspace libs (#506, from the nvlibs stage above) + the .so.1 sonames
+# PyNvVideoCodec dlopens. Harmless when decode_backend=cpu; required for nvdec_resident.
+ARG NVIDIA_DRIVER_VERSION
+COPY --from=nvlibs /nvlibs/ /usr/lib/x86_64-linux-gnu/
+RUN ln -sf "libnvcuvid.so.${NVIDIA_DRIVER_VERSION}" /usr/lib/x86_64-linux-gnu/libnvcuvid.so.1 \
```

```
+    && ln -sf "libnvidia-encode.so.${NVIDIA_DRIVER_VERSION}" /usr/lib/x86_64-linux-
gnu/libnvidia-encode.so.1 \
+    && ldconfig
+
 # --- WASB detector env (micromamba + conda-forge ? BSD-licensed, NO Anaconda ToS; P3a,
ADR-005) ---
 # Enforces ADR-005 (every dependency ? code, data, weights, AND the build toolchain ? stays
 # MIT/Apache/BSD): micromamba (BSD-3) + conda-forge (community) replaces Miniconda's Anaconda-
ToS-gated
 # `defaults` channels (the ?200-employee Terms-of-Service trap flagged in PACKAGING_PLAN.md
§3).
 # MAMBA_ROOT_PREFIX=$CONDA_DIR (=/opt/conda) keeps the env at $CONDA_DIR/envs/wasb so
$WASB_PYTHON below
-# is UNCHANGED. Same verified torch-1.11.0+cu113 WASB stack; the pip wheels are unaffected by
the switch.
+# is UNCHANGED.
+#
+# #506 modernized stack (validated end-to-end on a real L4, 2026-07-06): py3.10 + torch
2.6.0+cu124
+# (torch ?2 needs py?3.9; WASB HRNet ports clean ? cpu F1 0.576 ? the torch-1.11 baseline
0.582) +
+# PyNvVideoCodec (MIT ? the practical NVDEC path; TorchCodec needs a matching ffmpeg build and
could
+# not load) for decode_backend=nvdec_resident. wasb_infer.py carries the two version shims this
bump
+# needs (torch.load weights_only=False; the NumPy-2.0 np.Inf alias for WASB's tracker). numpy
is
+# UNPINNED (the old "numpy<1.24" was the torch-1.11 ABI pin). WASB-SBDT ships no
requirements.txt ?
+# the explicit list below is the (L4-validated) dependency closure its src/ imports need.
 ENV MAMBA_ROOT_PREFIX=/opt/conda
 RUN curl -sSL https://micro.mamba.pm/api/micromamba/linux-64/latest \
         | tar -xj -C /usr/local bin/micromamba \
-    && micromamba create -y -q -n wasb -c conda-forge python=3.8 \
+    && micromamba create -y -q -n wasb -c conda-forge python=3.10 \
     && git clone --depth 1 https://github.com/nttcom/WASB-SBDT.git "$WASB_REPO_DIR" \
     && "$CONDA_DIR/envs/wasb/bin/python" -m pip install -q \
-        torch==1.11.0+cu113 torchvision==0.12.0+cu113 --extra-index-url
https://download.pytorch.org/whl/cu113 \
-    && ( [ -f "$WASB_REPO_DIR/requirements.txt" ] && "$CONDA_DIR/envs/wasb/bin/python" -m pip
install -q -r "$WASB_REPO_DIR/requirements.txt" || true ) \
+        torch==2.6.0 torchvision==0.21.0 --index-url https://download.pytorch.org/whl/cu124 \
+    && "$CONDA_DIR/envs/wasb/bin/python" -m pip install -q PyNvVideoCodec \
     && "$CONDA_DIR/envs/wasb/bin/python" -m pip install -q \
-        hydra-core omegaconf pandas opencv-python-headless pillow scipy tqdm pyyaml matplotlib
"numpy<1.24"
+        hydra-core omegaconf pandas opencv-python-headless pillow scipy tqdm pyyaml matplotlib
numpy

 # --- Main application env ---
 WORKDIR /app
```

7. user (2026-07-07T04:16:02.715Z)

```
backend\pipeline\detectors\wasb_infer.py:83:    from hydra import compose, initialize
backend\pipeline\detectors\wasb_infer.py:405:    import cv2
backend\pipeline\detectors\wasb_infer.py:406:    from PIL import Image
backend\pipeline\detectors\wasb_infer.py:791:    import cv2
backend\pipeline\detectors\wasb_infer.py:817:    import cv2
backend\pipeline\detectors\wasb_infer.py:840:    import cv2
```

8. user (2026-07-07T04:16:37.969Z)

```
Structured output provided successfully
```